

Módulo 10 – Capítulo 09 – Sección 01

Taxonomía de costos: inferencia, almacenamiento, pipelines de datos y herramientas

Los costos de operar una plataforma de IA tienen una estructura significativamente diferente a los costos de operar aplicaciones web tradicionales, y entender esa diferencia es el prerequisite para gestionarlos con eficacia. En un servicio web convencional, el costo de infraestructura es dominado por compute (CPU, memoria) y almacenamiento, con una curva de crecimiento relativamente predecible en función del número de usuarios. En una plataforma de IA, la estructura de costos es más heterogénea: el costo de inferencia de LLMs puede dominar todos los demás costos combinados en aplicaciones de alta frecuencia, el costo de GPU para entrenamiento tiene picos pronunciados y períodos de inactividad que dificultan la predicción, y el costo del storage para modelos grandes y datasets de entrenamiento puede ser significativo aunque individualmente parezca pequeño.

La taxonomía completa de costos de una plataforma de IA incluye cuatro categorías principales. Los **costos de inferencia** son los más variables y frecuentemente los más grandes: llamadas a APIs de proveedores externos (OpenAI, Anthropic, Google Gemini) facturadas por tokens de input y output, o GPU time para modelos self-hosted cuyo costo depende del tamaño del modelo, el hardware disponible, y la eficiencia del sistema de batching. Para referencias actualizadas sobre precios específicos de instancias GPU o tokens de LLM, consultar las calculadoras de precios de AWS, GCP, Azure, OpenAI y Anthropic directamente —los precios evolucionan con suficiente frecuencia como para que cualquier valor fijo en este texto quede desactualizado en meses. Lo que sí es estructuralmente estable son los rangos de magnitud relativos: las instancias con GPUs de última generación (H100, A100) cuestan entre 5 y 15 veces más por hora que las instancias con GPUs de generación anterior, y los modelos de la tier premium de los proveedores cuestan entre 10 y 30 veces más por token que los modelos de la tier económica.

Los **costos de compute de entrenamiento** tienen la característica de ser episódicos: grandes gastos puntuales concentrados en los períodos de entrenamiento o fine-tuning, seguidos de períodos de costo cero. Esta estructura hace que las **instancias spot o**

preemptibles sean especialmente relevantes para training: AWS Spot Instances y GCP Preemptible VMs proveen la misma capacidad de GPU que las instancias on-demand a un descuento del 60-90%, a cambio de que el proveedor puede interrumpirlas con 2 minutos de preaviso cuando necesita esa capacidad para instancias on-demand. Para jobs de entrenamiento con checkpointing cada N minutos, la pérdida máxima de trabajo ante una interrupción es el trabajo del último checkpoint —tolerable para la mayoría de los training jobs de duración mayor a una hora.

Los **costos de almacenamiento** tienen una escala que suele sorprender a equipos sin experiencia previa con modelos grandes: un modelo de lenguaje de 7B parámetros en float16 ocupa aproximadamente 14 GB; un modelo de 70B parámetros ocupa 140 GB; un corpus de preentrenamiento de 100B tokens puede ocupar varios terabytes en formato tokenizado. En object storage, la diferencia de costo entre storage classes (Standard vs Infrequent Access vs Archive) puede ser de 5x a 20x para el mismo volumen de datos —automatizar la transición de datasets históricos a clases de almacenamiento más baratas es una optimización con ROI inmediato. El **feature store online** en Redis tiene un costo por GB-mes significativamente más alto que el object storage (típicamente en la escala de 10-50x), lo que hace que el TTL de las features online —cuánto tiempo se mantienen en Redis antes de eliminarse— sea una palanca de optimización importante.

Los **costos de MLOps tooling** son frecuentemente subestimados porque se amortiza entre todos los equipos usuarios. Las licencias de herramientas como Weights & Biases, las versiones enterprise de MLflow (via Databricks), Tecton, o herramientas de observabilidad especializadas tienen costos anuales que pueden ser significativos pero que divididos entre el número de equipos que los usan representan un costo por equipo razonable.

Categorías de costos y su estructura

- **Costos de inferencia externa:** facturados por tokens (input + output) en modelos de API; altamente variables según el volumen, el modelo seleccionado y el tamaño del contexto; los rangos de precios varían significativamente entre la tier premium y la tier económica de los proveedores; consultar calculadoras oficiales para valores actualizados.
- **Costos de compute de entrenamiento:** GPU hours en instancias cloud; usar spot/preemptible instances reduce el costo 60-90% para jobs tolerantes a interrupciones con checkpointing; las instancias con GPUs de última generación son 5-15x más caras por hora que las de generaciones anteriores.

- **Costos de almacenamiento:** object storage para datasets activos, storage de archivo para datasets históricos (5-20x más barato), Redis para feature store online (10-50x más caro que object storage por GB); automatizar la transición entre storage classes según el uso es una optimización con ROI inmediato.
- **Costos de MLOps tooling:** licencias de herramientas de experiment tracking, model registry enterprise, feature store managed, observabilidad; amortizables entre todos los equipos usuarios; a menudo subvalorados en la estimación inicial de costos de la plataforma.
- **Costos de la plataforma (overhead):** cluster de Kubernetes de control (nodos de sistema, ingress, monitoring); equipo de plataforma (el costo más grande y frecuentemente el más subestimado); estos costos son fijos independientemente del volumen de uso, y se amortiza mejor con mayor número de equipos consumidores.

Nota del Arquitecto: El primer paso para optimizar costos de IA es tener visibilidad: sin tagging de recursos por equipo y proyecto, y sin reportes de atribución de costos, es imposible identificar qué está generando el gasto y dónde están las oportunidades de optimización. La visibilidad de costos no es una iniciativa de FinOps avanzada: es el prerequisite de cualquier optimización, y puede implementarse con tagging de recursos y las herramientas nativas de cada cloud provider antes de necesitar herramientas especializadas.

La taxonomía de costos establece el mapa del territorio. La siguiente sección examina el mecanismo central de accountability financiero —chargeback y showback— que convierte esa taxonomía en responsabilidad por equipo y proyecto, creando los incentivos correctos para que los equipos de ingeniería adopten prácticas de eficiencia de costos.

Módulo 10 – Capítulo 09 – Sección 02

Chargeback y showback: asignar costos de plataforma a los equipos que los generan

Chargeback y showback son dos modelos de atribución de costos de infraestructura a los equipos consumidores que resuelven el mismo problema de visibilidad pero con diferentes consecuencias para el comportamiento. Sin ninguno de los dos, el costo de la infraestructura de IA aparece como un costo centralizado opaco —"infraestructura de plataforma: \$500,000 al año"— sin que nadie en la organización sepa cuánto cuesta el modelo de recomendaciones versus el modelo de detección de fraude versus el LLM de soporte al cliente. Esta opacidad elimina los incentivos para que los equipos optimicen su consumo: si el costo de usar modelos más caros o de hacer training jobs más frecuentes de lo necesario no se refleja en el presupuesto del equipo, no hay señal económica que incentive la eficiencia.

El **showback** es el primer paso: dar a cada equipo visibilidad de cuánto cómputo, almacenamiento y uso de LLMs está consumiendo, en términos de costo estimado, sin que ese costo tenga consecuencias presupuestarias directas para el equipo. El showback es informativo, no punitivo: "el equipo de NLP consumió \$15,000 en inferencia de GPT-4 este mes" es información que el equipo puede usar para decidir si optimizar, pero no se les cobra directamente desde su presupuesto de ingeniería. La experiencia de organizaciones como Airbnb y Stripe con la introducción gradual de showback es consistente: cuando los equipos ven por primera vez cuánto cuestan sus sistemas de IA, es común que descubran que el 60-70% del costo está generado por el 20% de las features, y que ese 20% tiene casos de uso de menor prioridad que el costo no justifica.

El **chargeback** va un paso más allá: el costo real consumido por cada equipo se carga efectivamente al presupuesto del equipo en los reportes financieros internos. El equipo de NLP ve en su presupuesto mensual que gastó \$15,000 en inferencia de GPT-4, y ese gasto reduce su presupuesto disponible para contratar, comprar herramientas, o hacer training jobs adicionales. El chargeback crea los incentivos económicos correctos: los ingenieros toman decisiones más conscientes sobre qué modelos usar, qué volúmenes de inferencia son necesarios, y si el caching o el tiering de modelos justifican la inversión de tiempo para

implementarlos. La buena práctica es implementar showback primero durante 3-6 meses para que los equipos se acostumbren a la visibilidad y ajusten sus patrones de consumo, y luego activar el chargeback cuando la cultura de responsabilidad de costos está establecida.

La implementación técnica del chargeback requiere instrumentación en todos los componentes de la plataforma. En Kubernetes, **Kubecost** (propietario) o **OpenCost** (open source, mantenido por la CNCF) calculan el costo de cada Pod, Deployment y namespace basándose en los precios reales de las instancias del cluster, con integración directa con AWS Cost Explorer o GCP Billing API para obtener precios actualizados. En el LLM Gateway, cada request registra el `project_id` y `team_id` con los tokens de input y output; un job de agregación diario calcula el costo total por proyecto usando la tabla de precios del proveedor y produce el reporte de showback o chargeback con granularidad de equipo y proyecto. En el almacenamiento, los cost allocation tags propagados a AWS Cost Explorer o GCP Billing permiten calcular el costo de almacenamiento por equipo con la misma granularidad.

Aspectos técnicos del chargeback para plataformas de IA

- **Resource tagging strategy:** convención obligatoria de tags en todos los recursos cloud: `team`, `project`, `environment`, `cost_center`; validado en el webhook de admisión de Kubernetes y en el CI/CD pipeline; sin tag, sin recurso.
- **Kubecost / OpenCost:** calcula el costo por namespace de Kubernetes basándose en precios reales de instancia (CPU, memoria, GPU); integrable con los sistemas de billing interno y con Grafana para visualización en el dashboard de la plataforma.
- **LLM cost attribution:** el LLM Gateway registra `project_id` y `team_id` en cada request; un job de agregación diario calcula el costo total usando la tabla de precios del proveedor y produce el reporte de showback/chargeback con desglose por modelo, project y team.
- **Internal billing system:** sistema que consolida datos de Kubecost, LLM Gateway, y storage billing para producir el estado de cuenta mensual por equipo; puede ser tan simple como un script que publica en Confluence o tan completo como un sistema interno de billing con integración a las herramientas de finanzas de la organización.
- **Budget alerts:** umbrales de gasto configurables por equipo que disparan notificaciones automáticas a Slack cuando el consumo alcanza el 80% del presupuesto mensual asignado, con datos granulares de qué proyecto está generando el gasto; la notificación debe llegar con tiempo suficiente para ajustar el comportamiento antes de superar el 100%.

Nota del Arquitecto: *Comenzar con showback antes de implementar chargeback permite a los equipos ajustar sus patrones de consumo sin la presión de consecuencias presupuestarias inmediatas, resultando en una adopción más orgánica de las prácticas de eficiencia de costos. Los equipos que descubren por sí mismos que están gastando más de lo esperado en un modelo que podría reemplazarse con uno más barato son más receptivos a optimizar que los equipos que reciben una factura interna inesperada el primer mes de chargeback.*

El chargeback efectivo convierte la responsabilidad de costos de IA en parte de la cultura de ingeniería de los equipos, no en una restricción impuesta desde FinOps central. La siguiente sección examina las estrategias técnicas de optimización de costos de inferencia que los equipos pueden implementar una vez que tienen visibilidad del gasto: batching, caching y tiering de modelos.

Módulo 10 – Capítulo 09 – Sección 03

Optimización de inferencia: batching, caché y selección de modelo por tier de calidad

La optimización del costo de inferencia es el área de mayor impacto económico en la mayoría de las plataformas de IA, porque el costo de inferencia escala linealmente con el volumen de uso mientras que el costo de entrenamiento es puntual. A diferencia de la optimización de entrenamiento —donde la principal palanca es el tamaño del job y el tipo de instancia— la optimización de inferencia opera sobre el ciclo completo de cada request individual, lo que significa que las mejoras se replican automáticamente en todos los requests subsiguientes sin esfuerzo adicional. Las tres estrategias con mayor impacto son el **batching** (agrupar múltiples requests para procesarlos juntos), el **caching** (reutilizar respuestas previas para requests similares), y el **tiering de modelos** (seleccionar el modelo más barato que sea suficientemente bueno para cada tipo de request).

El **continuous batching** (también llamado dynamic batching o in-flight batching) es la optimización más importante para el throughput de modelos self-hosted. Los servidores de inferencia como **vLLM** implementan continuous batching con **PagedAttention** —un mecanismo de gestión del KV cache inspirado en la paginación de memoria de sistemas operativos— que permite procesar múltiples requests simultáneamente sin desperdiciar capacidad de GPU entre la finalización de un request y el inicio del siguiente. El resultado es un aumento del throughput de hasta 23x respecto a servir requests de uno en uno, sin impacto significativo en el TTFT para cada request individual. Triton Inference Server también soporta dynamic batching con `preferred_batch_size` configurable. Para modelos self-hosted de producción, no habilitar continuous batching equivale a operar un avión con la mitad de los asientos vacíos en cada vuelo.

El **tiering de modelos** es la optimización más impactante para costo en aplicaciones que usan APIs externas. El principio es directo: no todos los requests necesitan el modelo más capaz y más caro. Un análisis de las queries de producción de muchas aplicaciones muestra que el 60-80% de los requests son tareas relativamente simples —clasificación, extracción de entidades, resumen de texto corto, generación de respuestas formulaicas— para las cuales un

modelo de la tier económica (GPT-4o-mini, Claude Haiku, Gemini Flash) produce resultados de calidad equivalente a una fracción del costo. El modelo de la tier premium (GPT-4o, Claude Opus, Gemini Pro) es necesario solo para el 20-40% de requests que requieren razonamiento complejo, generación creativa de alta calidad, o comprensión de contextos muy largos y complejos. Implementar un clasificador de complejidad de requests —que puede ser tan simple como una lista de palabras clave en el system prompt o tan sofisticado como un modelo de clasificación binaria— y usar ese clasificador para routing en el LLM Gateway puede reducir el costo total de inferencia en un 50-80% con degradación mínima de la calidad percibida.

La **compresión de prompts** es una optimización complementaria para contextos largos: técnicas como LLMLingua o Selective Context reducen el número de tokens en el prompt manteniendo la información relevante mediante extracción o compresión del texto de contexto. Para aplicaciones de RAG con contextos extensos —recuperación de múltiples documentos largos para responder una pregunta— la compresión del contexto puede reducir los tokens de input en un 20-40%, con reducción proporcional del costo. La **quantización** para modelos self-hosted —conversión de los pesos de float32 a INT8 (GPTQ, AWQ) o INT4— reduce los requerimientos de VRAM a la mitad y aumenta el throughput 1.5-2x con degradación de calidad inferior al 2% en la mayoría de las tareas, haciendo posible servir modelos más grandes en el mismo hardware.

Estrategias de optimización de costos de inferencia

- **Continuous batching (vLLM, Triton):** el motor de inferencia procesa múltiples requests en paralelo con PagedAttention para gestionar el KV cache; aumenta throughput hasta 23x vs serving secuencial; no habilitar continuous batching en modelos self-hosted es equivalente a subutilizar el GPU en un factor grande.
- **Semantic caching en el LLM Gateway:** embeddings + búsqueda vectorial para retornar respuestas cacheadas para prompts similares; efectivo en aplicaciones con consultas repetitivas (FAQ, generación templated); puede reducir costos 30-70% con threshold de similitud bien calibrado (ver Capítulo 07).
- **Model tiering:** clasificar requests por complejidad y enrutar a modelos de diferente tier; los modelos de tier económica cuestan entre 10 y 30 veces menos que los de tier premium con calidad equivalente para el 60-80% de las tareas; el clasificador de complejidad puede implementarse como regla en el gateway.
- **Prompt compression:** técnicas de compresión del contexto (LLMLingua, Selective Context) que reducen tokens de input 20-40% para prompts largos; especialmente

relevante para aplicaciones de RAG con contextos extensos y recurrentes.

- **Quantization para modelos self-hosted:** INT8 (GPTQ, AWQ) o INT4 quantization reduce requerimientos de VRAM a la mitad y aumenta throughput 1.5-2x con degradación de calidad < 2% en la mayoría de las tareas; primera optimización a evaluar antes de escalar hardware.

Nota del Arquitecto: La optimización del costo de inferencia no es una tarea única que se hace una vez: es un proceso continuo de perfilado del uso real en producción (qué tipos de requests, qué modelos, qué tamaños de contexto) y calibración de las estrategias de optimización basadas en esos datos. Los datos de producción del LLM Gateway —disponibles en el audit log descrito en el Capítulo 07— son la fuente de información correcta para identificar qué optimizaciones tendrán mayor impacto en el caso de uso específico de la organización.

Las estrategias de optimización de inferencia pueden reducir el costo de operación de una plataforma de IA en un factor de 3x a 10x respecto a una configuración sin optimización, con el caching y el tiering de modelos como las contribuciones más grandes en la mayoría de los casos. La siguiente sección examina cómo estas estrategias se integran con las prácticas de FinOps de la organización para crear un sistema coherente de gestión de costos de IA.

Módulo 10 – Capítulo 09 – Sección 04

FinOps para IA: integración del modelo de costos con las prácticas de FinOps de la organización

FinOps (Financial Operations) es la práctica de ingeniería de costos cloud que une los equipos de finanzas, tecnología y negocio para optimizar el gasto en cloud de forma continua —no como proyecto puntual sino como proceso operacional sostenido. La FinOps Foundation define tres fases que se repiten cíclicamente: **Inform** (visibilidad y atribución de costos), **Optimize** (reducción activa del gasto sin degradar capacidad), y **Operate** (cultura continua de responsabilidad de costos con retroalimentación entre finanzas, ingeniería y negocio). Las plataformas de IA requieren una extensión específica de este modelo para gestionar los costos específicos de inferencia, entrenamiento y almacenamiento de modelos.

La fase de **Inform** para plataformas de IA requiere instrumentación adicional respecto al cloud nativo convencional. El tagging de recursos que proveen herramientas como Kubecost o el Cost Explorer del cloud provider es suficiente para atribuir el costo de compute y storage por equipo. El costo de inferencia de LLMs —que viene de APIs externas con facturación por tokens— requiere instrumentación adicional en el LLM Gateway: cada request registra el equipo, el proyecto, el modelo usado, los tokens de input y output, y el costo calculado. La agregación de estos datos en un sistema de reporting unificado —que puede ser una hoja de cálculo avanzada en los primeros estadios o un sistema de billing interno completo en organizaciones maduras— produce el cuadro de mando que finanzas necesita para entender el gasto de IA de la organización.

Las **unit economics de IA** son el artefacto más valioso que la fase de Inform puede producir para las decisiones de negocio: el costo por unidad de valor creada. "Costo por query de soporte al cliente resuelto con IA: \$0.12" o "Costo por usuario activo mensual en el feature de recomendaciones de IA: \$0.45" son métricas que los product managers pueden usar para evaluar el ROI de los features de IA y priorizar inversiones. Sin unit economics, la conversación sobre el valor de la IA es cualitativa; con ellas, es financiera. Calcular las unit economics requiere conectar los datos de costo del LLM Gateway y Kubecost con las métricas

de negocio de la aplicación (queries resueltas, usuarios activos) en un data pipeline que actualice los números periódicamente.

La fase de **Optimize** para plataformas de IA combina las estrategias técnicas de la sección anterior (tiering, batching, caching) con estrategias financieras de compra de capacidad. La más impactante es la compra de **capacidad reservada** (Reserved Instances en AWS, Committed Use Discounts en GCP) para la carga base predecible de los endpoints de producción: los endpoints de producción tienen patrones de tráfico relativamente predecibles (pico diario en horario laboral, bajo en madrugada) que justifican comprometer capacidad mínima a cambio de descuentos del 40-60% respecto al precio on-demand. La capacidad variable por encima del baseline se atiende con instancias on-demand o spot según la tolerancia a interrupciones del workload.

La fase de **Operate** es donde el FinOps para IA se convierte en cultura de ingeniería: los equipos conocen el costo unitario de sus decisiones técnicas, los dashboards de FinOps son parte del ritual de revisión semanal del equipo, y las decisiones de arquitectura de IA incluyen estimaciones de costo como parte del análisis de trade-offs. Esta cultura no emerge espontáneamente: requiere que el equipo de plataforma publique los dashboards de costo de forma accesible, que los Product Managers incluyan los unit economics en sus reviews de features, y que la dirección de ingeniería haga visible que el costo de infraestructura de IA es una responsabilidad compartida, no solo una preocupación del equipo de finanzas.

Prácticas de FinOps aplicadas a plataformas de IA

- **Unit economics de IA:** calcular y publicar el costo por unidad de valor de negocio: costo por predicción, costo por query LLM, costo por usuario activo diario que usa features de IA; habilita decisiones de ROI explícitas y priorización de optimizaciones.
- **Reserved capacity para GPU y compute:** Reserved Instances (AWS) o Committed Use Discounts (GCP) para la carga base predecible de producción; descuento del 40-60% vs on-demand para compromisos de 1-3 años; spot/preemptible para training y workloads interruptibles.
- **Rightsizing de clusters:** análisis mensual de utilización real de recursos (CPU, memoria, GPU) de los pods del training cluster; reducir las resource requests y limits a lo que realmente se usa para reducir el desperdicio de capacidad pagada pero no utilizada.
- **Spot instance strategy para training:** AWS Spot o GCP Preemptible para training jobs con checkpointing cada N minutos; reducción de costo del 60-90%; requiere que el pipeline de training esté instrumentado con checkpointing automático y retry logic.

- **Cost anomaly detection:** alertas automáticas cuando el gasto diario de un equipo supera 2x su gasto diario promedio de los últimos 30 días; investigación proactiva antes de que el gasto escale a fin de mes sin que nadie lo haya notado.

Nota del Arquitecto: FinOps para IA requiere que los equipos de ingeniería adopten responsabilidad por los costos que generan sus decisiones técnicas: qué modelo seleccionan, qué tamaño de contexto envían, qué frecuencia de reentrenamiento configuran. Esta responsabilidad no emerge espontáneamente de dashboards de visibilidad —requiere que el costo sea parte de las métricas de equipo que se revisan en el standup y en el planning, con la misma importancia que la latencia o el error rate.

FinOps para IA convierte la visibilidad de costos en acción: las prácticas de la fase Inform proveen los datos, las de la fase Optimize implementan las mejoras, y las de la fase Operate garantizan que las mejoras no se revierten cuando el equipo cambia o cuando el volumen de uso crece. La siguiente sección examina los controles técnicos de presupuesto que previenen que el gasto escale de forma inesperada antes de que el sistema de alertas tenga tiempo de reaccionar.

Módulo 10 – Capítulo 09 – Sección 05

Presupuestos y alertas: controles de gasto antes de que los costos escalen

La diferencia entre una plataforma de IA gestionada financieramente y una que acumula sorpresas en la factura mensual es la presencia de controles de gasto preventivos: mecanismos que detectan el problema antes de que escale, no reportes que documentan el daño después. El escenario de riesgo clásico es el loop infinito en producción: una aplicación cliente con un bug que llama al endpoint de LLM en bucle puede consumir el presupuesto diario completo de un equipo en cuestión de minutos, y sin un sistema de hard stop automático, el equipo y el equipo de plataforma se enteran solo cuando llega la factura —o cuando el sistema de monitoreo de costos lanza una alerta, pero para entonces el gasto ya ocurrió. Los controles de presupuesto técnicos convierten las alertas reactivas en guardrails proactivos que interrumpen el gasto anormal antes de que alcance una escala crítica.

La arquitectura de controles de presupuesto se estructura en tres capas que operan a diferentes velocidades. La **capa de aplicación** opera en el LLM Gateway: cada equipo tiene un presupuesto diario configurado en USD, el gateway lleva un contador incremental en Redis que se actualiza con cada request procesado, y cuando el contador supera el umbral el gateway devuelve HTTP 429 con el header `Retry-After` indicando cuándo se reiniciará el contador. Esta capa tiene latencia de decisión de microsegundos porque el costo acumulado está en memoria —no requiere consultar un servicio externo. La **capa de infraestructura** opera en los controles del cloud provider: AWS Budgets y GCP Budget Alerts con alertas configuradas al 50%, 80%, y 100% del presupuesto mensual, con acciones automáticas que pueden detener instancias o escalar a cero cuando se superan los umbrales críticos. Esta capa cubre el gasto de infraestructura que no pasa por el LLM Gateway: instancias del training cluster, almacenamiento, y servicios cloud adicionales. La **capa de orquestación** opera en el nivel de los pipelines de entrenamiento: antes de lanzar un training job, el sistema de orquestación estima el costo total basándose en el tipo de instancia, el número de GPUs, y la duración estimada del job, y requiere aprobación explícita del responsable técnico del equipo si el costo estimado supera un umbral configurable.

El **sistema de respuesta gradual** de tres niveles es la práctica más efectiva para combinar alerta temprana con acción proporcional. Al 80% del presupuesto diario, el sistema envía una notificación a Slack en el canal del equipo con el desglose de gasto por proyecto y modelo: información suficiente para que el equipo investigue si hay un patrón anormal de consumo antes de que sea un problema. Al 100%, el gateway activa el **model downgrade automático**: las requests que normalmente irían al modelo premium son redirigidas al modelo económico para el mismo equipo, manteniendo el servicio activo pero con calidad ligeramente reducida. Al 110%, el gateway activa el **hard stop**: todas las llamadas del equipo reciben HTTP 429 hasta que el presupuesto se restablece al día siguiente o hasta que el equipo contacta al equipo de plataforma para solicitar una extensión de emergencia. El nivel del 110% existe para cubrir el caso donde el volumen de requests llega tan rápido que el sistema no tiene tiempo de reaccionar entre la alerta del 100% y el impacto financiero.

La **estimación de costos de training jobs** es el control preventivo más importante para los jobs de entrenamiento de larga duración. Un training job de fine-tuning de un modelo de 7B parámetros en 8 GPUs puede costar varios cientos a miles de dólares según la duración; lanzar jobs sin una estimación previa del costo total puede resultar en que el presupuesto mensual del equipo se consuma en un único experimento mal configurado. El sistema de estimación usa la configuración del job (instance type, número de instancias, duración estimada basada en el tamaño del dataset y la configuración de epochs) para calcular un costo estimado que se muestra al usuario antes del submit. Si el costo supera el presupuesto disponible del equipo, el sistema bloquea el lanzamiento y solicita aprobación del responsable técnico o ajuste de la configuración.

Controles técnicos de presupuesto para plataformas de IA

- **Daily spend limits en el LLM Gateway:** presupuesto diario configurable por equipo en USD con contador incremental en Redis; el gateway rechaza requests con HTTP 429 y header `Retry-After` cuando el acumulado supera el límite; decisión de sub-millisecond basada en estado en memoria.
- **Tres niveles de respuesta automatizada:** alerta informativa en Slack al 80% (tiempo para investigar); model downgrade automático al modelo económico al 100% (mantiene el servicio); hard stop completo al 110% (bloqueo hasta reinicio de contador o aprobación de extensión de emergencia).
- **AWS Budgets / GCP Budget Alerts:** alertas nativas del cloud provider a múltiples umbrales del presupuesto mensual de infraestructura; configurables para disparar

acciones automáticas (detener instancias EC2/GCE, enviar a SNS/Pub-Sub) adicionales a las notificaciones.

- **Estimación de costo de training jobs:** antes de lanzar un training job, el sistema estima el costo total basándose en instance type, número de instancias y duración estimada; requiere aprobación explícita si el costo estimado supera el presupuesto disponible del equipo.
- **Anomaly detection de gasto:** alerta automática cuando el gasto diario de un equipo supera la media más 2 desviaciones estándar del histórico de 30 días; indicativo de un bug (loop infinito en producción) o de un cambio no planificado en el volumen de uso que requiere investigación.
- **Cost circuit breaker para autoescalando:** mecanismo que congela el número de réplicas de los pods de serving cuando el costo proyectado del día supera el presupuesto del equipo, evitando que el autoescalando agrave el problema cuando el origen del gasto es un pico de tráfico anormal.

Nota del Arquitecto: Los controles de presupuesto más efectivos son los que operan sin intervención humana para detener el problema, y al mismo tiempo proveen información suficiente para que el equipo entienda qué ocurrió. Un hard stop sin contexto genera pánico; un hard stop con el desglose de gasto por proyecto, modelo y horario permite al equipo identificar el bug y resolverlo en minutos. El diseño de los mensajes de error del sistema de presupuesto merece la misma atención que el diseño de los controles en sí.

Los presupuestos y alertas técnicas completan la arquitectura de gestión de costos de la plataforma: la taxonomía de costos establece el mapa, el chargeback crea los incentivos, las optimizaciones de inferencia reducen el gasto base, las prácticas de FinOps generan la cultura, y los controles de presupuesto previenen que los incidentes de costo escalen. La siguiente sección cierra el capítulo con la perspectiva de síntesis sobre la gestión financiera de plataformas de IA como práctica de ingeniería de largo plazo.

Módulo 10 – Capítulo 09 – Sección 06

Cierre: el costo de operar una plataforma de IA es predecible si se gobierna correctamente

La imprevisibilidad del gasto en una plataforma de IA es casi siempre una señal de déficit en la instrumentación, no de variabilidad inherente e ingobernable del uso de IA. Cuando cada token consumido, cada GPU-hora procesada, y cada gigabyte almacenado está atribuido en tiempo real a un equipo y proyecto, las anomalías de costo se detectan en minutos, no en semanas. La organización que recibe sorpresas en su factura cloud de IA cada mes no tiene un problema de variabilidad del uso: tiene un problema de visibilidad. La organización que puede predecir su gasto de IA con una precisión del 10-15% mensual tiene instrumentación completa, chargeback activo, y un proceso continuo de revisión de unit economics. La diferencia entre estas dos organizaciones no es tecnológica —las herramientas están disponibles— sino de disciplina operacional: decidir que el costo de IA es una métrica de ingeniería con la misma importancia que la latencia o la disponibilidad.

La secuencia de madurez en la gestión de costos de IA sigue cuatro pasos que este capítulo ha recorrido en orden: **visibilidad primero** (taxonomía de costos y tagging completo de recursos), **responsabilidad segundo** (chargeback y showback que crean los incentivos correctos en los equipos consumidores), **optimización tercero** (batching, caching, tiering de modelos, compresión de prompts, quantización), y **cultura permanente** (FinOps embebido en los rituales semanales de los equipos, unit economics en las reviews de features). La secuencia importa: intentar optimizar antes de tener visibilidad es optimizar sin saber qué importa; intentar crear cultura de costos antes de tener chargeback es pedirle a los equipos que se preocupen por una restricción que no sienten.

La optimización de costos de inferencia es también un **diferencial competitivo** que trasciende el ahorro presupuestario. Una plataforma de IA cuyo costo por query LLM es 5x menor que el de una configuración naive —gracias al continuous batching, el semantic cache bien calibrado, y el tiering de modelos— puede ofrecer más capacidad experimental a los equipos de AI Engineering con el mismo presupuesto, puede justificar el uso de features de IA en casos de uso con menor ROI unitario, y puede reducir el precio de los productos basados

en IA de la organización. Las optimizaciones de inferencia no son solo "reducir costos": son "aumentar la capacidad de experimentación e innovación dentro del mismo presupuesto", que es exactamente la misión del equipo de plataforma.

Para recordar

La madurez en la gestión de costos de IA se mide por la capacidad de responder en tiempo real a la pregunta "¿cuánto cuesta esta feature de IA por usuario activo diario?": si la respuesta requiere semanas de análisis, el gobierno de costos es insuficiente. Si la respuesta está disponible en el dashboard de la plataforma con un clic, el sistema ha alcanzado el estado que permite tomar decisiones financieras de IA con la misma agilidad con que se toman decisiones de producto.

"Efficiency is doing things right; effectiveness is doing the right things."

— Peter F. Drucker, padre del management moderno. La distinción de Drucker entre eficiencia y efectividad captura con precisión la doble responsabilidad del FinOps para IA: optimizar el costo de operar los modelos (eficiencia) es solo valiosa si la organización está invirtiendo en los modelos y features que generan valor real de negocio (efectividad). Un sistema de IA perfectamente optimizado que resuelve el problema equivocado es el ejemplo más costoso de eficiencia sin efectividad.